

A Lean 4 Certificate of Furber et al (CSL 2026) Interpreting Lambda Calculus in Domain-Valued Random Variables

Lars Warren Ericson¹

¹Catskills Research Company

¹<https://github.com/catskillsresearch/scott2026>

¹lars.ericson@catskillsresearch.com

October 2, 2026

ORCID: 0000-0001-8299-9361

Primary Category: cs.LO (Logic in Computer Science)

Secondary Categories: math.LO (Logic); cs.PL (Programming Languages)

Palomar Registration:

<https://palomar-registry.org/entry?id=PALOMAR-2026-09-28-000003&version=1>

Abstract

This note records a Lean 4 / Mathlib formalization of Furber, Mardare, Panangaden, and Scott’s CSL 2026 paper *Interpreting Lambda Calculus in Domain-Valued Random Variables*. The paper develops Boolean-valued set theory and domain theory and interprets the λ -calculus via domain-valued random variables. The sorry-free `Scott2026/` library (~38,000 lines) covers V^A , the internal Engeler model, Theorems 26 and 30, Corollary 34, Lemma 35 / Proposition 36, the measure algebra $A(X) = \Sigma/\mathcal{N}$, Propositions 39–42, Theorem 43, and Proposition 44. Continuous-lattice background is vendored from `scott1972` (`vendor/scott1972`). The deliberate `sorry` appears only in the Mathlib-only `Challenge.lean`; the Comparator hole is `cs12026.proposition_36_i` is a definition without a proof hole, and `Solution.lean` re-exports kernel-checked proofs. There are no project axioms beyond Mathlib’s classical footprint. Dana Scott gave the author the paper; this formalization does not claim the paper’s authors’ participation or endorsement. Lean was written by AI agents under the author’s direction and review. Proof summaries include short Lean fragments; the full library is indexed with links to <https://github.com/catskillsresearch/scott2026>.

1 Introduction

Boolean-valued models of set theory were introduced so that independence results—most famously the continuum hypothesis—could be read as statements about names and Boolean truth values rather than as metatheoretic constructions of forcing extensions [Sco67, Bell, Jech]. Scott later asked whether the same idea could serve *semantics*: if the Boolean algebra A is a measure algebra, then an A -valued set is a random variable taking values in a domain, and the internal language of V^A becomes a language of almost-sure statements about higher-type computation.

The CSL 2026 paper carries that program through for the untyped λ -calculus. It rebuilds, internally, the ingredients of a reflexive domain: A -posets, Scott continuity, the Engeler graph model $\mathcal{P}^A(\check{E})$, Church numerals, and a many-one comparison of subsets of $\mathbb{N}$ defined by λ -terms. The application (Theorem 43) produces two subsets of $\mathbb{N}$ that are incomparable under

that λ -definable preorder, using independent fair coins as the underlying negligibility space. Proposition 44 records that one cannot shortcut the argument by treating $L^0(X; \mathcal{P}(Y))$ as a continuous dcpo: when $A(X)$ is non-atomic, it is not.

This formalization follows the paper’s numbered statements in dependency order. Weaker or special-case Lean theorems keep their original names; paper-type statements are introduced beside them (`theorem_30_va` beside `theorem_30`, `lemma_35_of` beside Engeler `lemma_35`, and so on). The working source is `sources/Scott2026.pdf`, with a searchable transcription in `sources/Scott2026_vision.md`. A related long version is [arXiv:2112.06339](https://arxiv.org/abs/2112.06339).

Three methodological choices are worth stating up front.

1. **Extensional ZFC boundary.** Ground sets are read through Mathlib’s `ZFSet` quotient. Raw `PSet` representatives remain an implementation device, used only where structural recursion needs them.
2. **Nontrivial Boolean algebras.** The paper’s distinctions ($\perp \neq \top$, numeral separation, strictness of $\text{Oid}(D)$) have no intended meaning in the one-element algebra. Lean states `[Nontrivial A]` exactly where those distinctions appear.
3. **Measure algebra as $\Sigma/\mathcal{N}$.** The paper’s $A(X)$ is the quotient of *measurable* sets by null sets (`MeasureAlgebra`). An all-sets presentation (`NegligibilitySpace.measurableRep`) is retained but is strictly stronger than Borel coin space can inhabit.

The formalization is **classical**. Every completed proof reports the standard mathlib footprint `[propext, Classical.choice, Quot.sound]`.

2 Scope and layout

The sorry-free development is about 38,000 lines across the modules under `Scott2026/`, plus the vendored Scott 1972 continuous-lattice library. The deliberate compared proof hole occurs only in the Mathlib-only `Challenge.lean`. Its compared declarations are Theorem 43 and its exact Engeler-oracle statement definition, `cs12026` and `proposition_36_i`. The full internal Corollary 34 remains kernel-checked as `corollary_34`, without a weaker Mathlib-only proxy. The remaining paper-type theorems are kernel-checked in `Scott2026/` and re-exported by `Solution.lean`.

Module cluster	Role
<code>VA, ExtensionalVA, Oid, Setoid, RelFun</code>	Boolean-valued names, A -setoids, mixing
<code>InternalDomain, InternalInterp, Internal</code>	Internal Definition 19 and the Theorem 26
<code>Eval*</code>	evaluator
<code>Theorem26, Theorem30Internal,</code>	Internal interpretation and Engeler in V^A
<code>Corollary34</code>	
<code>Domain, Lambda, Interp, Lemma35General,</code>	Ground reflexive dcpos, Church calculus,
<code>Prop36</code>	oracles
<code>Random, Coin</code>	Measure algebra, L^0 , G_X , Propositions 42–44
<code>Basic</code>	Re-exports and inventory comments
<code>Challenge / Solution</code>	Mathlib-only challenge statements vs sorry-free proofs

`Scott1972` is compiled from `vendor/scott1972 (srcDir; pin a198b6e, see vendor/FROZEN.txt)`, not as a Lake path/git dependency.

3 How the proofs use Mathlib

The paper rebuilds a large fragment of Boolean-valued set theory and domain theory. Mathlib supplies the algebraic and measure-theoretic substrate; the paper-specific language (A -names, internal Scott continuity, Church combinators, G_X) is developed in `Scott2026/`.

Order and Boolean algebra. Complete Boolean algebras, Heyting implication, and complete lattices come from `Mathlib.Order.CompleteBooleanAlgebra` and `Mathlib.Order.CompleteLattice`. Internal predicates (`isFunctionB`, `isScottContinuousB`, `isReflexiveDcpo` `B`) are A -valued terms in that algebra, not Prop-valued class-instance searches.

ZFC presentations. `Mathlib.PSet / ZFSet (Mathlib.SetTheory.ZFC)` give a well-founded model of sets. We use `PSet` only for recursion (Church numerals as `PSet.omega`, pairing codes) and expose `ZFSet / checkExt` at the paper boundary, with representation-independence lemmas so two equivalent pre-sets cannot create a spurious failure of strictness.

Domain theory. Scott-open sets, way-below, and the injective extension operator `scottExtend` (Scott 1972, Props 2.11 / 3.8) come from the vendored `Scott1972` library, which itself sits on Mathlib topology (`IsEmbedding`, `Continuous`, Scott topology). Lemma 35(ii) applies `scottExtend` to the discrete numeral subspace and feeds the extension to `ReflexiveDcpo.lam`.

Measure theory. Fair coins are Mathlib Bernoulli measures on `Bool` (extended by `Measure.infinitePi` to 2^ω and by `Measure.prod` to $2^\omega \times 2^\omega$). Nullity, countable unions, and `toMeasurable` are the usual `MeasureTheory` lemmas. The paper algebra `MeasureAlgebra`, parameterized by a measure μ , is a `Quotient` of $\{s \text{ // MeasurableSet } s\}$ by $\mu(s \triangle t) = 0$, with a `CompleteBooleanAlgebra` instance under `[IsFiniteMeasure  $\mu$ ]` (countable-chain-condition essential suprema).

Combinatorics of syntax. Capture-avoiding substitution needs `[Infinite Var]`. Countability of closed λ -terms for Theorem 43 uses an injective `Nat.pair` encoding (`Mathlib.Data.Nat.Pairing`).

What Mathlib does *not* provide, and what this library therefore builds, is the internal language of V^A : Boolean-valued membership and equality on names, the `Oid` equivalence, the relational evaluator of Definition 25, and the $A(X)$ -poset L^0 .

4 Proof dependency structure

The paper’s numbered results are not independent modules. The Lean import graph follows the published order: extensional names, then the internal evaluator, then Engeler in V^A , then numerals and oracles, then the measure-algebra example.

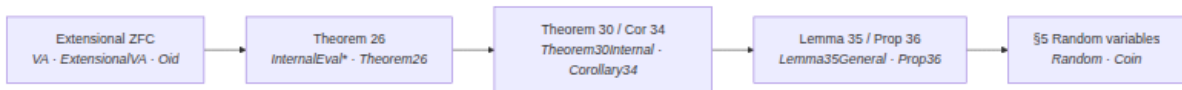


Figure 1: High-level dependency order of the formalized paper results.

5 Theorem inventory

Paper names and Lean names. Weaker or special-case theorems keep their original names beside the paper-type theorems.

Paper	Paper-type Lean name	Weaker / special-case name
Proposition 27	<code>proposition_27 /</code> <code>proposition_27_finite_*</code> <code>wrappers</code>	<code>finite_subsets_countable</code>

Paper	Paper-type Lean name	Weaker / special-case name
Theorem 26	<code>theorem26Full</code> / <code>theorem26Pure</code> / <code>theorem26Pure_sound</code>	<code>theorem_26</code> (Engeler carrier)
Theorem 30	<code>theorem_30_va</code> / <code>theorem_30_internalModel</code>	<code>theorem_30</code> (canonical-carrier)
Corollary 34	<code>corollary_34</code>	<code>corollary_34_check</code>
Lemma 35	<code>lemma_35_of</code> / <code>lemma_35_i_of</code> / <code>lemma_35_ii_of</code>	<code>lemma_35</code> (Engeler)
Proposition 36	<code>proposition_36_of</code>	<code>proposition_36</code> (Engeler)
Proposition 42	<code>proposition_42_algebra</code>	<code>proposition_42</code> (raw coin Measure)
Theorem 43	<code>theorem_43_paper</code>	<code>theorem_43</code> (external oracles)
Proposition 44	<code>proposition_44_coin</code> / <code>_measure</code> / <code>_dcpo</code>	<code>proposition_44</code> (all-sets algebra)

Further named results: `definition_25` / `interp`, `lemma_31`, `definition_32`, `proposition_33`, `churchNotN` / `churchTest` / `churchWithNumerals`, `MeasureAlgebra` / `coinAlgebra` / `coinS1` / `coinS2`, `G_X_measure`, `proposition_39_measure`, `proposition_40_measure`, `lemma_41_measure` / `lemma_41_algebra`, `not_isAtomic_coinAlgebra`.

Compared declarations in `Challenge.lean`: `cs12026` (Theorem 43) and `proposition_36_i`. The full Corollary 34 is not selected because faithfully restating its `AName`-based internal predicates in a Mathlib-only Challenge would require replacing them with a weaker proxy. The proof of `cs12026` uses `cs12026_capstones` (`theorem26Full`, `corollary_34`, `theorem_43_paper`). Representative challenge statements:

Lean 4 source

```
theorem cs12026 : exists T_1 T_2 : Set Nat,
  not proposition_36_i T_1 T_2 /\ not proposition_36_i T_2 T_1 := by
  sorry

def proposition_36_i (S_1 S_2 : Set Nat) : Prop :=
  exists M : Lam Nat, MapsNumerals M /\
    exists d_1 d_2 : Set Nat,
      engelerIsOracle d_1 S_1 /\
      engelerIsOracle d_2 S_2 /\
      forall n,
        engelerApp engelerPair d_2
          (engelerInterpClosed (M.app (churchNumN n))) =
          engelerApp engelerPair d_1 (engelerNumeral n)
```

The simpler membership equivalence $n \in S_1 \leftrightarrow \text{mapsNumeralsFun } M \text{ hM } n \in S_2$ is the derived lemma `proposition_36_i_mem`, not the selected definition.

6 Proof notes

This section records strategy where the mechanization has to say more than the published text, and shows a short Lean fragment when the library proof is short. Longer constructions are cited by file.

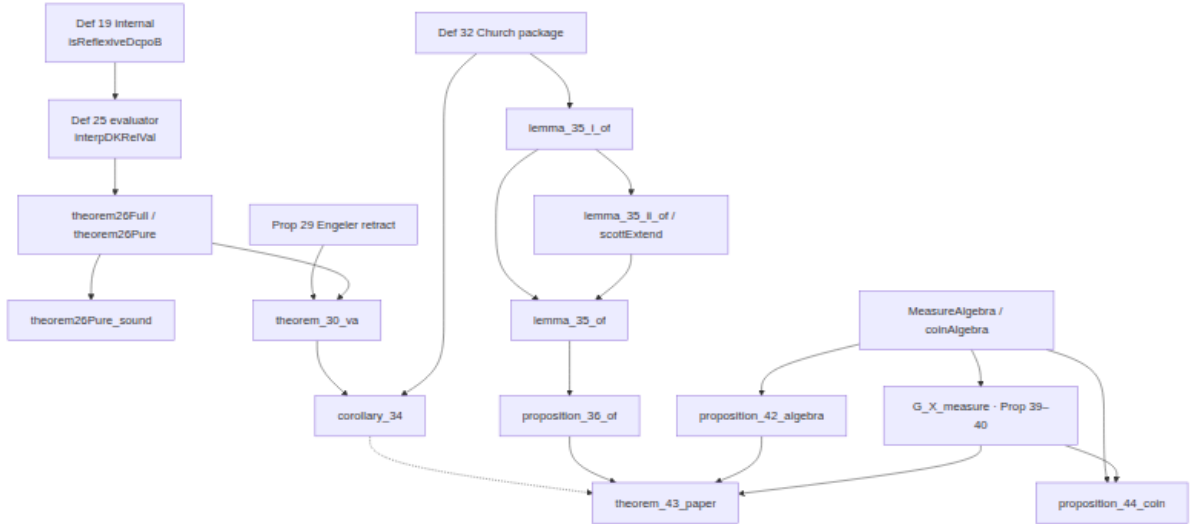


Figure 2: Internal interpretation, Engeler model, numerals, and the random-variable capstone (Theorems 26–43).

6.1 Boolean-valued set theory and Oid

Jech’s lemmas on mixing, fullness, and Δ_0 reflection are `jech_lemma_14_18`, `theorem_1_iii`, and `jech_lemma_14_21`. The categorical equivalence between A -setoids and extensional names is `setAEquivSetoidR`. Completeness, totality, and strictness of $\text{Oid}(\mathcal{P}^A(X))$ are `theorem_17_va_*` / `proposition_28_va_*`. These are the substrate for every later internal statement: an internal function name is an A -valued relation that is single-valued and total at Boolean degree $\top$, then packaged as a `SetoidFHom` by Definition 16.

6.2 Theorem 26 — internal interpretation

Definition 25 interprets $\Lambda(D, \text{Var}, \mathfrak{K})$ by recursion on terms, in the internal language. The difficulty the paper leaves implicit is that abstraction must survive *fuzzy* environment updates: the body is interpreted along a pointwise Scott-continuous family, not a crisp substitution. The library therefore proves a fundamental lemma quantifying over `IsPointwiseFamily` environments (`InternalEvalFamily`, `InternalEvalComplete`), identifies application and abstraction body graphs as Scott-continuous maps into the internal function space C , and only then packs an evaluator name `interpDKGraph` (`InternalEvalPack`).

`theorem26Full` is then Definition 16 applied to that name:

Lean 4 source

```
noncomputable def theorem26Full
  (? : InternalReflexiveModel (A := A))
  (V K : AName.{u} A) (hK : subsetB K ?.D = top)
  (hV : (oid V).IsTotal)
  (? : RelFun (oid V) (oid ?.D)) :
  SetoidFHom (oid (lamDKB ?.D V K)) (oid ?.D) :=
theorem26Full0fFunctionName ?.D V K
  (InternalReflexiveModel.interpDKGraph ? V K hK hV ?)
  (InternalReflexiveModel.interpDKGraph_isFunctionB ? V K hK hV ?)
  ?.complete
```

Soundness `theorem26Pure_sound` is the full `LamEq` theory (reflexivity, symmetry, transitivity, congruence, ξ , capture-avoiding β , α). The extra hypothesis actually needed is `Oid Separated V` (distinct variable keys are Boolean-unequal), together with `[Infinite V.idx]`

for `Lam.substCA`. The Engeler-only `theorem_26` on `Set ℕ` is kept as a weaker packaging. Full proof: `Scott2026/LambdaModels/DomainTheory/InternalEvalComplete.lean`, `InternalEvalPack.lean`, `Theorem26.lean`.

6.3 Theorem 30 and Corollary 34 — Engeler in V^A

Proposition 29’s `app/lam` pair is internalized as names `engelerFun` and `engelerLamB` on $D = \mathcal{P}^A(\omega)$. The retract `fun ∘ lam = idC` uses Scott continuity of $G \in C$ on the directed family of finite subsets, plus membership of ground finite sets in the checked ω . The six-tuple of Definition 19 then has Boolean value $\top$:

The packaging of Corollary 34 is a conjunction, so the library proof is a tuple of already-proved conjuncts.

Lean 4 source

```
theorem corollary_34 [Nontrivial A] :
  isReflexiveDcpoB engelerD engelerR engelerC engelerQ
    engelerFun engelerLamB = top /\
  isContinuousLatticeSubsetB engelerD = top /\
  isBaseSubsetB (pfinB (checkExt PSet.omega)) engelerD = top /\
  Corollary34Check /\
  (eqB (child? (interpClosedVA churchTrue))
    (child? (interpClosedVA churchFalse)) = False) /\
  (forall n m, eqB (child? (interpClosedVA (churchNum n)))
    (child? (interpClosedVA (churchNum m)))) = top -> n = m) :=
<theorem_30_va, isContinuousLatticeSubsetB_powerB _,
  isBaseSubsetB_pfinB_powerB _, corollary_34_check,
  eqB_interpClosedVA_churchTrue_churchFalse,
  fun n m h => churchNum_interpClosedVA_injective h>
```

(`[Nontrivial A]` is required for `checkExt` and for the last two conjuncts.) Full proof: `Scott2026/LambdaModels/Engeler/Theorem30Internal.lean`, `Corollary34.lean`.

6.4 Lemma 35 — discrete numerals and oracles

The paper derives closed tests $\mathbf{m}^?$ from Definition 32’s `if`, `pred`, and `0?`, then uses Scott-open separators and (for part (ii)) injectivity of continuous lattices. In Lean, `churchNotN` and `churchTest` live on $\mathbb{N}$ so that `interpClosed_sound_full` applies (`[Infinite Var]`). Transfer of the λ -equations is one rewrite:

Lean 4 source

```
theorem interpClosed_churchNotN_true {D : Type u} [CompleteLattice D]
  (R : ReflexiveDcpo D) :
  R.app (interpClosed R churchNotN) (interpClosed R churchTrueN) =
    interpClosed R churchFalseN := by
  rw [ <- interpClosed_app ]
  exact interpClosed_sound_full (Var := Nat) R churchNotN_true
```

N numeral injectivity is the paper’s function argument: if $\llbracket c_m \rrbracket = \llbracket c_n \rrbracket$ then `fun`($\llbracket m^? \rrbracket$) cannot send one to $\top$ and the other to $\perp$ (`numeral_inj_of_churchTest`). Part (ii) embeds the numeral subspace, notes that discreteness makes $g(\llbracket c_n \rrbracket) = \chi_A^D(n)$ continuous, and extends by `scott Extend` (Scott 1972). The retract identity `lemma_35_ii_of_extension` turns `lam  $\bar{g}$` into the oracle. Full proof: `Scott2026/LambdaModels/DomainTheory/Lambda.lean`, `Scott2026/LambdaModels/Oracles/Lemma35General.lean`.

6.5 Proposition 36 — λ -definable many-one comparison

On a reflexive *continuous* lattice with Church numerals, paper (i) (some oracles) and (ii) (all oracles) are equivalent because Lemma 35(ii) supplies at least one pair. The library proof is exactly that pair of implications; `ManyOneLe` (an arbitrary $f : \mathbb{N} \rightarrow \mathbb{N}$) is kept separate, and the converse `ManyOneLe` $\rightarrow$ `proposition_36_i` is not claimed (`exists_nat_fun_not_lambda_definable`).

Lean 4 source

```
theorem proposition_36_of {D : Type*} [CompleteLattice D]
  (R : ReflexiveDcpo D)
  (hbool : interpClosed R churchTrueN != interpClosed R churchFalseN)
  (hcont : Scott1972.ContinuousLattice.IsContinuousLattice D)
  (S_1 S_2 : Set Nat) :
  proposition_36_i_of R hbool S_1 S_2 <->
    proposition_36_ii_of R hbool S_1 S_2 :=
  <proposition_36_ii_of_i_of R hbool,
    proposition_36_i_of_ii_of R hbool hcont>
```

6.6 Measure algebra, G_X , and Proposition 42

`MeasureAlgebra` quotients measurable sets. The Boolean value of a measurable event is $\perp$ exactly when the event is null:

Lean 4 source

```
theorem mk_eq_bot [IsFiniteMeasure ?] {s : Set X}
  {hs : MeasurableSet s} :
  mk ? s hs = (False : MeasureAlgebra ?) <-> ? s = 0 := by
  rw [ <- mk_bot, mk_eq_iff, ae_empty_iff]
```

Equation (5) is `coinS1 n = mk (D1 n true)` (and likewise `coinS2`). Proposition 39–40 and Lemma 41 are restated on this algebra as `proposition_39_measure`, `proposition_40_measure`, `lemma_41_measure`; the un-suffixed names remain the all-sets `AssociatedAlgebra` forms. Proposition 42’s algebra form is four Boolean values equal to $\perp$: no finite standard preimage of S_1 or S_2 , and neither $S_1 = \check{f}^{-1}(S_2)$ nor the swap. Each conjunct is a Mathlib null-set calculation already in `proposition_42`, lifted by `mk_eq_bot`.

Lean 4 source

```
theorem proposition_42_algebra (f : Nat -> Nat) :
  (( bigcup K : Finset Nat, ? n, (coinS1 n ? checkSet K (f n)) sqcap
    (checkSet K (f n) ? coinS1 n) ) = False) /\
  (( bigcup K : Finset Nat, ? n, (coinS2 n ? checkSet K (f n)) sqcap
    (checkSet K (f n) ? coinS2 n) ) = False) /\
  ((? n, (coinS1 n ? coinS2 (f n)) sqcap (coinS2 (f n) ? coinS1 n) = False) /\
  ((? n, (coinS2 n ? coinS1 (f n)) sqcap (coinS1 (f n) ? coinS2 n) = False) :=
  <iSup_eq_bot.mpr fun K => coinS1_finite_preimage_eq_bot f K,
    iSup_eq_bot.mpr fun K => coinS2_finite_preimage_eq_bot f K,
    coinS1_preimage_S2_eq_bot f, coinS2_preimage_S1_eq_bot f>
```

The finite- K join is over `Finset  $\mathbb{N}$` , not the full internal $\mathcal{P}_{\text{fin}}^A(\tilde{\omega})$. Full proof: `Scott2026/RandomVariables/Random.lean`, `Coin.lean`.

6.7 Theorem 43 — incomparable λ -degrees

The paper applies Theorem 1 to Lemma 35(ii) *inside* V^A , transports along G_X^{-1} , and intersects countably many conull sets. The library proof `theorem_43_paper` follows that *mathematical*

recipe with one honest substitution of tools: oracles are fiberwise ground Lemma 35(ii) (**chi Oracle** on the bit-sets), mixed as L^0 maps and pushed through **G_X_measure**, rather than a direct **theorem_1_iii** witness on an internalized Lemma 35 formula. Application is the existing Engeler operation **engelerAppA**. After Proposition 42 makes each numeral-mapping M null, countability of $\text{Lam } \mathbb{N}$ (**lamEncode_injective**) and **measure_univ = 1** produce a point x ; $T_i = \{n \mid a_i(x) \cdot \llbracket c_n \rrbracket = \llbracket \top \rrbracket\}$ and Proposition 36 give both non-reducibility conclusions.

The statement is short; the construction of the conull set is not, so this note does not copy the body. The capstone type matches the library proof:

Lean 4 source

```
theorem theorem_43_paper :
  exists T_1 T_2 : Set Nat, not proposition_36_i T_1 T_2 /\ not proposition_36_i T_2 T_1
```

After **exists_mem_paperGoodSet** the library defines T_i as the set of n with $a_i(x) \cdot \llbracket c_n \rrbracket = \llbracket \top \rrbracket$ and finishes with **proposition_36_i** in **Scott2026/RandomVariables/Coin/Proofs/CoreCont.lean**. The external-oracle theorem **theorem_43** is unchanged and has the same type.

6.8 Proposition 44 — L^0 is not a continuous dcpo

On a complete lattice the paper’s “continuous dcpo” is the existing way-below-sup predicate (**IsContinuousDcpo** aliases **IsContinuousLattice**). Transport along G_X reduces the claim to: the function space of a non-atomic complete Boolean algebra is not a continuous lattice. Fair-coin **coinAlgebra** is non-atomic because any positive class splits along a first-sequence bit (or else determines that sequence and is null). Full proof: **proposition_44_coin** in **Coin.lean**; the AssociatedAlgebra form **proposition_44** is kept.

7 Source fidelity

Working source: **sources/Scott2026.pdf**. Vision transcription: **sources/Scott2026_vision.md** (produced by **scripts/ocr_pdf_pipeline.sh**).

7.1 Foundational convention

We read the paper’s ground sets extensionally, in ZFC. The Lean formalization therefore exposes ground-set parameters through **Mathlib’s ZFSet**; **PSet** is used only as an implementation-level well-founded presentation behind proved representation-independence lemmas. Boolean names use the quotient domain **AName.Dom**, and checked sets use **checkExt**, so duplicate pre-set presentations cannot create spurious failures of strictness.

We also believe the paper implicitly assumes that the complete Boolean algebra A is non-trivial. Statements that distinguish Boolean truth values or require $\perp \neq \top$ therefore state **[Nontrivial A]** explicitly. This is necessary: for the one-element Boolean algebra all Boolean equalities simultaneously have values $\top$ and $\perp$, so the paper’s numeral-separation and strictness conclusions cannot have their intended meaning.

7.2 Recorded divergences

- **proposition_42_algebra** finite- K is a **Finset** join (the paper’s fuzzy **pfinB** exists-quantifier is not fully internalized).
- **theorem_43_paper** uses fiberwise Lemma 35(ii) mixed in $L^0/\mathbf{G_X_measure}$, not a direct **theorem_1_iii** hookup; **theorem_43** remains the external-oracle form.

- All-sets `NegligibilitySpace.measurableRep / ofMeasure hN` is stronger than paper $\Sigma/\mathcal{N}$; paper $A(X)$ is `MeasureAlgebra`.
- Engeler-only `lemma_35 / proposition_36 / proposition_42 / theorem_43` keep weaker or special-case names.
- The Challenge compares Theorem 43 and its Proposition 36(i) relation. All paper-facing library capstones remain kernel-checked.

8 Build

The repository pins Lean **v4.35.0-rc3** (`lean-toolchain`) and Mathlib **v4.35.0-rc3** (`lakefile.toml`).

```
lake exe cache get
lake build
bash scripts/build_arxiv_pdf.sh
```

Regenerate `arxiv_with_code.md` when `arxiv.md` or listed sources change: `bash scripts/generate_arxiv_with_code.sh`.

9 Palomar registry

The compared statements are registered with Palomar as **PALOMAR-2026-09-28-000003**, version 1 [1]. That registry entry records a Mathlib-only statement of this formalization, by Lars Warren Ericson. The compared declarations are Theorem 43 (`cs12026`) and the Engeler-oracle relation `proposition_36_i`.

10 Acknowledgments

Thanks to **Prof. Dana Scott** (Carnegie Mellon University) for giving the author the CSL 2026 paper.

10.1 AI-assisted development

Lean in this repository was written by AI agents under the author’s direction and review. The human author retains sole responsibility for the mathematical content, the formalization route, and every formal claim. **No large language model is listed as a co-author.**

We gratefully acknowledge assistance from the following tools:

- **GPT-5.6** [2] — OpenAI’s GPT-5.6 family in Cursor (Sol variant) for Lean 4 / mathlib formalization, `lake build` repair, and long-running agent loops on `Scott2026/`. Generated Lean was provisional until it compiled under the pinned toolchain.
- **Anthropic Claude Fable 5** [3] — selective use in Cursor for the heaviest proof work and long-horizon formalization. Every emitted proof term was checked by the Lean kernel.
- **Anthropic Claude Opus 5** [4] — day-to-day formalization and proof-engineering in Cursor: inventory and narrative maintenance, module wiring, and medium-to-hard Lean obligations where the proof strategy was already fixed.
- **Cursor Grok 4.6** [5] — primary SpaceXAI / Cursor agent for Lean 4 / mathlib formalization, `lake build` repair, vision-OCR transcription, drafting this narrative (`arxiv.md`), and tracking the formalized inventory.

- **Cursor Composer 2.5** [6] — routine multi-step work: module scaffolding, dependency-ordered wiring of `Scott2026/`, documentation, and medium proof obligations where the strategy was already fixed.

List of Figures

1	High-level dependency order of the formalized paper results.	3
2	Internal interpretation, Engeler model, numerals, and the random-variable capstone (Theorems 26–43).	5
3	The four appendix tables, in the order of the paper.	12

11 References

- [1] Palomar Registry. A Lean 4 certificate of Furber et al (CSL 2026) interpreting lambda calculus in domain-valued random variables, 2026. PALOMAR-2026-09-28-000003, version 1. <https://palomar-registry.org/entry?id=PALOMAR-2026-09-28-000003&version=1>.
- [2] OpenAI. *GPT-5.6* (Sol variant as integrated in Cursor). Model documentation, <https://cursor.com/docs/models/gpt-5-6-sol>, 2026. Accessed 2026.
- [3] Anthropic. *Claude Fable 5*. Announcement, <https://www.anthropic.com/news/claude-fable-5-mythos-5>; system card, <https://www-cdn.anthropic.com/2f9323abbcc4abe219577539efe19a623c9ca2bd/Claude%20Fable%205%20%20Claude%20Mythos%205%20System%20Card.pdf>; Cursor model page, <https://cursor.com/docs/models/claude-fable-5>, 2026. Accessed 2026.
- [4] Anthropic. *Claude Opus 5*. Announcement, <https://www.anthropic.com/research/claude-opus-5>; model documentation as integrated in Cursor, <https://cursor.com/docs/models/claude-opus-5>, 2026. Accessed 2026.
- [5] SpaceXAI and Anysphere, Inc. *Grok 4.6*. Official model card, <https://media.x.ai/v1/website/card-7f81d41b.pdf>; developer documentation, <https://docs.x.ai/developers/models/grok-4.6>; Cursor model page, <https://cursor.com/docs/models/grok-4-6>; Cursor announcement, <https://cursor.com/blog/grok-4-6>, 2026. Accessed 2026.
- [6] Anysphere, Inc. *Composer 2.5*. Model announcement and documentation, <https://cursor.com/blog/composer-2-5>; model card as integrated in Cursor, <https://cursor.com/docs/models/composer-2-5>, 2026. Accessed 2026.

12 License and source PDF

Original Lean and author-written docs: Apache-2.0. `sources/Scott2026.pdf` and `sources/JechSetTheory2003.pdf` are **not** Apache-2.0; see NOTICE and `sources/README.md`.

A Lean module index

Subsection	Paper	What the table covers
A.1 Boolean-valued set theory	§2	Every module in the section.
A.2 Boolean-valued setoids	§3	Every module under <code>Setoids/</code> .
A.3 Boolean-valued models of the untyped λ -calculus	§4	One table, with divider rows for domain theory (4.1), the Engeler model (4.2), and injective spaces and oracles (4.3).
A.4 Random variables	§5	Every module under <code>RandomVariables/</code> , then the capstone <code>Paper.lean</code> .

Figure 3: The four appendix tables, in the order of the paper.

Each row is one Lean module. The summary is taken from that file’s module note or the declaration it introduces. `Scott2026.lean` is at the top level of the repository. Next to it is the directory `Scott2026`, and underneath that is `Basic.lean`. The folder name is in the section

header. That folder is omitted from the printed module name; the link still opens the full path. When a module has submodules, they are listed and then each is given its own subsection. Sources: <https://github.com/catskillsresearch/scott2026>.

A.1 Boolean-valued set theory `BooleanValuedSetTheory`

Section 2 of the paper.

This module has submodules

- The Boolean-valued universe `VA`

Module	Contents
<code>BooleanLogic.lean</code>	Boolean-valued propositional and first-order connectives. Furber–Mardare–Panangaden–Scott, CSL 2026, §2: a complete Boolean algebra A interprets the connectives of propositional logic; completeness interprets $\forall$ as a meet and $\exists$ as a join. Validity means Boolean value $\top$.
<code>VA.lean</code>	The Boolean-valued universe V^A . Furber–Mardare–Panangaden–Scott, CSL 2026, §2, and Jech, <i>*Set Theory*</i> (2003), Chapter 14: names, Boolean values of membership and equality, the substitution laws (Lemma 14.16), mixing (14.18), fullness (14.19 = CSL Theorem 1(iii)), and Δ_0 invariance (14.21 = CSL Theorem 2).
<code>ExtensionalVA.lean</code>	Extensional domains of Boolean-valued names. The paper defines an A -name as a partial function whose domain is a set of names. <code>AName</code> uses an indexed presentation, so its raw index type can list Boolean-equal children more than once.
<code>RawPowerStrict.lean</code>	Regression test for raw presentation indices. Isolated counterexample: a pre-set that lists $\emptyset$ twice admits two different valuations on $\text{dom}(P^A(\text{check } Y))$ that are Boolean-equal at $\top$. This is not a paper counterexample: <code>PSet</code> presentations are implementation details, while the paper works with extensional ZFC sets.

A.1.1 The Boolean-valued universe `VA`

This module has submodules

- Boolean names `AName`
- Bounded formulas `DOFormula`
- Set-theoretic formulas `SetFormula`
- Proofs `Proofs`

Module	Contents
<code>SetFormula.lean</code>	Defines ‘ <code>SetFormula</code> ’.

Module	Contents
AName.lean	An A-name: a function from a small index type into $V^A \times A$.
DOFormula.lean	Δ_0 formulas with n free variables (bounded quantifiers only).
HomName.lean	A name that is a function $X \rightarrow_{_} A$ Y at Boolean value 1.
funB.lean	$\{F \in P^A(X \times_{_} A Y) \mid \text{isFunctionB } F \text{ } X \text{ } Y\}^A$.
prodB.lean	Cartesian product $X \times_{_} A Y$.
pairB.lean	$\{x, y\}^A$.
opairB.lean	Kuratowski ordered pair $(x, y)^A = \{\{x\}^A, \{x, y\}^A\}$.
singletonB.lean	$\{x\}^A$.
powerB.lean	$P^A(X)$: names with the same domain as X and values $\leq X(t)$.
sepB.lean	Separation $\{x \in X \mid \Phi(x)\}^A$.
isFunctionB.lean	Boolean value of “ $F \subseteq X \times_{_} A Y$ is a functional total relation”.
isSingleValuedB.lean	Single-valued: $(x, y_1) \in F \wedge (x, y_2) \in F \rightarrow y_1 = y_2$.
isTotalB.lean	Total: $x \in X \rightarrow \exists y, (x, y) \in F$.
canonVal.lean	Defines ‘canonVal’: A.
worITE.lean	Boolean value of a proposition, via classical decidability.
choiceAxiom.lean	Defines ‘choiceAxiom’: SetFormula 0.
wellOrderAxiom.lean	R well-orders X (free variables 0 = R, 1 = X).
opairMemF.lean	$\langle a, b \rangle \in r$.
isOpairF.lean	$p = \langle a, b \rangle$ (Kuratowski).
isSingletonF.lean	$z = \{a\}$ as $\forall w (w \in z \leftrightarrow w = a)$.
isUPairF.lean	$z = \{a, b\}$.
colRename.lean	Parameter shift for Collection: $v, u, \text{params} \mapsto v, u, Y, X, \text{params}$.
collectB.lean	Collection witness: one fullness realizer per child of X.
collectionAxiom.lean	Defines ‘collectionAxiom’: SetFormula n.
compB.lean	Relation composition $g \circ f \subseteq X \times_{_} A Z$.
eqLeibnizAxiom.lean	$\forall x \forall y (x = y \rightarrow (\varphi(x) \rightarrow \varphi(y)))$.
eqLeibnizRenameX.lean	Defines ‘eqLeibnizRenameX’: $\text{Fin } (n + 1) \rightarrow \text{Fin } (n + 2)$.
eqLeibnizRenameY.lean	Defines ‘eqLeibnizRenameY’: $\text{Fin } (n + 1) \rightarrow \text{Fin } (n + 2)$.
extensionalityAxiom.lean	Defines ‘extensionalityAxiom’: SetFormula 0.
finsetB.lean	Finite enumeration as an A-name.
homB.lean	$\text{Set}_{_} A(X, Y)$: function names at Boolean value 1, modulo $\ F = G\ = 1$.
nameSetoid.lean	Boolean equality at value 1 is an equivalence of names.
idB.lean	Identity relation on X: $\{(x, x)^A \mid x \in X\}$.
infinityAxiom.lean	Defines ‘infinityAxiom’: SetFormula 0.

Module	Contents
insertB.lean	Insert x into the name y.
isFiniteB.lean	Defines ‘isFiniteB’: A.
isInductiveB.lean	Boolean value of “X is inductive”: $\emptyset \in X$ and successor-closed.
succB.lean	Von Neumann successor $n \cup \{n\}$.
leastIdx.lean	Defines ‘leastIdx’: A.
pOpair.lean	Kuratowski ordered pair in PSet.
pairingAxiom.lean	Defines ‘pairingAxiom’: SetFormula 0.
pfin.lean	Finite subsets of X, presented by enumerations of elements of X.
pfinEnum.lean	Finite enumeration as a pre-set.
pfinB.lean	$P_fin^A(X)$ via separation on $P^A(X)$.
powerAxiom.lean	Defines ‘powerAxiom’: SetFormula 0.
regularityAxiom.lean	Defines ‘regularityAxiom’: SetFormula 0.
restrictName.lean	Canonical copy of S with domain $\text{dom}(X)$.
sepRename.lean	Parameter shift for Separation: $z, \text{params} \mapsto z, Y, X, \text{params}$.
separationAxiom.lean	Defines ‘separationAxiom’: SetFormula n.
unionAxiom.lean	Defines ‘unionAxiom’: SetFormula 0.
unionB.lean	Union name: domain is the disjoint union of the children’s domains.
wellOrderB.lean	Well-order name of X by least WellOrderingRel-index of a child.
worLe.lean	Defines ‘worLe’: A.
worDisj.lean	Defines ‘worDisj’: A.
worPredSup.lean	Defines ‘worPredSup’: A.

Boolean names [AName](#)

Module	Contents
child.lean	Children of a name.
idx.lean	Index type of a name (Jech: domain).
meas.lean	Lex measure $(\rho(x) \sqcup \rho(y), \rho(x), \rho(y))$ used to define $\ \in \ $ and $\ = \ $.
rank.lean	Jech rank $\rho(x)$: least strict upper bound of the ranks of the children.
measLt.lean	Defines ‘measLt’: $\text{Ordinal}.\{u\} \times \text{Ordinal}.\{u\} \times \text{Ordinal}.\{u\} \rightarrow \text{Ordinal}.\{u\} \times \text{Ordinal}.\{u\} \times \text{Ordinal}.\{u\} \rightarrow \text{Prop}$.
memEq.lean	From $\text{eqB } x \ y$, one recursive call is $\text{memB } (\text{child } y) \ x$ (arguments swapped).
val.lean	Boolean membership degrees of the children.

Module	Contents
fullness.lean	Proves ‘fullness’: $\exists a, \varphi a = \bigsqcup x, \varphi x$.
eqBLaws.lean	Jech Lemma 14.15: equality is reflexive with Boolean value 1.
mix.lean	Jech Lemma 14.18: mix of an antichain of names.
check.lean	Declares ‘check’, ‘check_mk’, ‘check_val’.

Bounded formulas [D0Formula](#)

Module	Contents
bval.lean	Boolean value of a Δ_0 formula at an A-name assignment.
consName.lean	Fin.cons on names, with a constant motive so both sides stay AName.
consPSet.lean	Fin.cons on pre-sets, with a constant motive.
realize.lean	Classical satisfaction of a Δ_0 formula in PSet.
iInfFinSucc.lean	Proves ‘iInf_fin_succ’: $\text{Fin } n, f \text{ i.succ}$.

Set-theoretic formulas [SetFormula](#)

Module	Contents
bval.lean	Boolean value of a formula of $\mathfrak{L}_{\text{Set}}(V^A)$ at a name assignment.
or.lean	Defines ‘or’: $\text{SetFormula } n$.
iff.lean	Defines ‘iff’: $\text{SetFormula } n$.
implies.lean	Defines ‘implies’: $\text{SetFormula } n$.
inst.lean	Substitute free variable 0 by the free variable t.
rename.lean	Rename free variables. Under a binder, index 0 stays bound.
lift.lean	Weaken: the new variable is 0, old free variables shift.

Proofs [Proofs](#)

Module	Contents
Core.lean	Internal set constructions (CSL 2026, §2 after Theorem 1).
CoreCont.lean	Theorem 2 consequences: check commutes with finite set formers.
CoreContCont.lean	$\mathfrak{L}_{\text{Set}}(V^A)$ and CSL Theorem 1(i)–(ii). The paper’s language of set theory with constants from V^A is the first-order language with $\in$ and $=$, evaluated at an assignment of A-names. Bounded Δ_0 formulas remain D0Formula. Unbounded quantifiers are SetFormula.ex and SetFormula.all , interpreted as joins and meets (CSL §2).
CoreContContCont.lean	Jech 14.27: every name is well-orderable in V^A .

Module	Contents
--------	----------

A.2 Boolean-valued setoids [Setoids](#)

Section 3 of the paper.

This module has submodules

- A-posets [APoset](#)
- A-setoids [ASetoid](#)

Module	Contents
ASetoid.lean	An A-valued setoid: a set with a symmetric, transitive A-valued equality (paper §3, before Definition 4). Reflexivity is not assumed.
APoset.lean	Definition 11: an A-poset. Equality is recovered as the symmetrization of $\leq$.
Setoid.lean	Boolean-valued setoids and posets (CSL 2026, §3). An A-setoid is a set equipped with an A-valued partial equivalence relation (Definitions 4–11, Lemma 12).
RelFun.lean	Relational and functional maps of A-setoids (Definitions 8–10, 13).
Categories.lean	Categories of Boolean-valued setoids. This module bundles the object types suppressed in Definitions 8 and 9 and installs the category structures using Mathlib’s category library.
SetoidFObj.lean	A separately bundled A-setoid, used as an object of SetoidF_A .
SetoidRObj.lean	A bundled A-setoid, used as an object of SetoidR_A .
PowerSet.lean	A-valued power sets (paper §2 constructions and Definitions 14–17). We work with A-valued subsets of a ground type X (functions $X \rightarrow A$). This is the fragment of V^A used for $\mathcal{P}^A(X)$ and for check-sets X.
Oid.lean	Oid : from A-sets to A-setoids (CSL 2026, Definitions 14–16). Definition 14 turns a name $X \in V^A$ into an A-setoid on $\text{dom}(X)$, Definition 15 turns a name S with $\ S \subseteq X\ = 1$ into a predicate on Oid(X) , and Definition 16 turns a function name $F : X \rightarrow_{\text{A}} Y$ into a relational function $\text{Oid}(X) \rightarrow \text{Oid}(Y)$.
OidEssential.lean	Essential surjectivity of Oid . The paper gives no proof of essential surjectivity.
SetCategory.lean	The category of Boolean-valued sets. This module bundles Set_A , lifts identity and relational composition through the name quotient, and packages Definition 16 as a Mathlib functor.
SetAObj.lean	Bundled objects of Set_A .

A.2.1 A-posets [APoset](#)

Module	Contents
StrictIso.lean	Strict isomorphism of A-posets: a bijection strictly preserving $\leq$.

A.2.2 A-setoids [ASetoid](#)

Module	Contents
Predicate.lean	Definition 7: a predicate on an A-setoid.
StrictIso.lean	Definition 5: a bijection strictly preserving A-valued equality.

A.3 Boolean-valued models of the untyped λ -calculus [LambdaModels](#)

Section 4 of the paper, in the order domain theory, the Engeler model, oracles.

This module has submodules

- Domain theory (section 4.1) [DomainTheory](#)
- The Engeler model (section 4.2) [Engeler](#)
- Injective spaces and oracles (section 4.3) [Oracles](#)

A.3.1 Domain theory (section 4.1) [DomainTheory](#)

This module has submodules

- Untyped λ -calculus [Lambda](#)
- Interpretation of λ -terms [Interp](#)
- Internal domain [InternalDomain](#)
- Internal evaluation [InternalEval](#)
- Completed internal evaluation [InternalEvalComplete](#)
- Packed internal evaluation [InternalEvalPack](#)

Module	Contents
IsDcpo.lean	A dcpo in the paper's exact sense: every nonempty directed subset has a least upper bound.
ReflexiveDcpo.lean	Definition 19: a reflexive dcpo, specialized to complete lattices.
ReflexiveDcpo19.lean	The dcpo of Scott-continuous maps, ordered pointwise.
Domain.lean	Domain theory background (CSL 2026, §4.1) and Proposition 27. §4.1 uses the order-theoretic way-below relation. The 1972 topological form lives in <code>Scott1972.ContinuousLattice</code> and implies this one.

Module	Contents
Valuation.lean	Definition 25: a valuation is a partial function $\text{Var} \rightarrow D$, represented as a Finset-supported total map. Only values on domain are used.
Lambda.lean	Untyped λ -calculus (Definitions 20, 23 and Example 21).
Interp.lean	Interpretation of pure λ -terms in a reflexive dcpo (Definition 25). Furber–Mardare–Panangaden–Scott, CSL 2026, Definition 25, following [4, Definition 5.4.2]. This is the ground interpretation on ReflexiveDcpo (Definition 19). There is no constant clause, no tag 3, and no $\Lambda(D, \text{Var}, \mathfrak{K})$.
AValuedReflexiveDcpo.lean	Theorem 30 data: an A-valued reflexive dcpo recorded as the poset $\ \subseteq\ $, application, abstraction, and the retract at Boolean value $\top$ for maps determined by finite sets. This is not Definition 19 inside V^A .
InterpConst.lean	Definition 25 with constants. The paper cites Barendregt for this recursion and its pure equational soundness. This file gives the four defining equations directly in Lean for the repository’s complete-lattice presentation of a reflexive dcpo.
InterpVA.lean	Theorem 26, pure-term fragment, on the Theorem 30 carrier. Furber–Mardare–Panangaden–Scott, CSL 2026, Theorem 26, restricted to $\Lambda(\text{Var})$ (no constants / tag 3) and the Engeler carrier of Theorem 30 (CanonicalPower Idx of check ω).
InterpConstVA.lean	Definition 25 for constant-bearing terms. This extends the existing Engeler-carrier interpretation from pure Lam to the full Definition 20 syntax. Constants are interpreted by a supplied map into the domain.
InternalDomain.lean	Module ‘InternalDomain’.
Proposition28.lean	Proposition 28. This module joins the syntactic $\mathfrak{L}_{\text{Set}}$ formulas with their semantic Boolean values and packages all clauses of Proposition 28. For the checked case we use checkExt, the extensional presentation of check whose domain has one index per extensional set.
ReflexiveVA.lean	Definition 19 inside V^A . These are the Boolean values of the order-theoretic clauses in Definition 19. Functions are represented by their internal graphs. The function-space name C is required extensionally to contain exactly the Scott-continuous self-maps of D, and its order is required to be the pointwise order.
InternalInterp.lean	Relational infrastructure for internal interpretation. This file develops the dependencies of the Definition 25 recursion. In particular, an internal inclusion $X \subseteq Y$ induces the relational function used to interpret constants from X as elements of Y.
InternalEval.lean	Internal Definition 25 evaluation. This file develops the valuation-parametric recursion omitted from the paper’s proof of Theorem 26.
InternalReflexiveModel.lean	The internal reflexive-dcpo data and the external setoid hypotheses of Theorem 26.

Module	Contents
InternalEvalFamily.lean	Parametricity of the internal evaluator. The abstraction case of Theorem 26 requires induction over a whole Boolean-valued family of environments. This module proves the extensional transport part of that strong induction.
InternalEvalComplete.lean	Completion of internal evaluation. This module supplies the degree-local normalization of abstraction body graphs, the directed-image calculus, fixed-argument evaluation, and the pointwise-supremum graph of a directed family in C.
InternalEvalPack.lean	Term induction and the Theorem 26 evaluator name. This module closes Definition 25 by induction on Lam DK, assembles the internal evaluator graph, and proves it is an internal function $\text{lamDKB } D \ V \ K \rightarrow D$.
Theorem26.lean	Theorem 26: internal interpretation. The Definition 25 evaluator name is assembled in InternalEvalPack and shown to be an internal function $\text{lamDKB } D \ V \ K \rightarrow D$. The four clauses are <code>interpDKGraph_var/const/app/abs</code> (and their <code>_val</code> unfoldings) at the term's <code>lamDKVal</code> extent.

Untyped λ -calculus [Lambda](#) This module has submodules

- λ -terms [Lam](#)
- Proofs [Proofs](#)

Module	Contents
Lam.lean	Definition 20: pure λ -terms over a set of variables (no domain constants).
churchFalse.lean	Defines ‘churchFalse’: $\text{Lam } (\text{Fin } 2)$.
churchFalseN.lean	Defines ‘churchFalseN’: $\text{Lam } \mathbb{N}$.
churchFalseN'.lean	false with binders 1,2, used inside 0?.
churchId.lean	The identity combinator $\lambda \text{id. id}$ used by Kleene pred.
churchIf.lean	The Church if combinator $\lambda b. \lambda t. \lambda e. b \ t \ e$, using a third name.
churchIfN.lean	$\text{if} = \lambda b \ t \ e. b \ t \ e$.
churchIsZero.lean	$0? = \lambda n. n \ (\lambda x. \text{false}) \ \text{true}$.
churchTrue.lean	Church Booleans on two distinct names (Definition 32 / Proposition 33).
churchTrueN.lean	Defines ‘churchTrueN’: $\text{Lam } \mathbb{N}$.
churchTrueN'.lean	true with binders 1,2, used inside 0?.
churchIter.lean	$F^n \ X$.
churchNotN.lean	Church negation $\lambda b. b \ \perp \ \top$ on $\mathbb{N}$.
churchNum.lean	Church numeral n as $\lambda f. \lambda x. f^n \ x$ on $\text{Fin } 2$.

Module	Contents
churchNumBind.lean	Church numeral with chosen binders (used to avoid capturing shift).
churchNumN.lean	Church numeral n as $\lambda f. \lambda x. f^n x$ on $\mathbb{N}$.
churchPred.lean	Kleene predecessor $\lambda n f x. n (\lambda g h. h (g f)) (\lambda u. x) (\lambda id. id)$.
churchPredBase.lean	Kleene base $\lambda u. x$ with free $x = 2$.
churchShift.lean	Kleene shift $\lambda g h. h (g f)$ with free $f = 1$.
churchShiftIter.lean	Defines ‘churchShiftIter’.
churchSucc.lean	$\text{succ} = \lambda n f x. f (n f x)$.
churchTest.lean	Defines ‘churchTest’.

λ -terms [Lam](#)

Module	Contents
FreeFor.lean	Declares ‘FreeFor’, ‘freeFor_var’, ‘freeFor_app’.
fv.lean	Free variables (Definition 20).
IsInductive.lean	Example 21: the inductive clauses for $\Lambda(\text{Var})$.
pickFresh.lean	Defines ‘pickFresh’: Var .
size.lean	Term size, used to justify capture-avoiding substitution.
subst.lean	Declares ‘subst’, ‘subst_var’, ‘subst_app’.
substNaive.lean	Declares ‘substNaive’, ‘size_substNaive_var’.
substCA.lean	Defines ‘substCA’.
vars.lean	All names occurring in a term, free or bound.

Proofs [Proofs](#)

Module	Contents
Core.lean	Definition 32 combinators on $\mathbb{N}$. $\text{Fin } 2$ / $\text{Fin } 3$ stubs above are unchanged. Capture-avoiding identities need [Infinite Var], so the paper package is stated on $\mathbb{N}$.
CoreCont.lean	Definition 32(iii): predecessor.

Interpretation of λ -terms [Interp](#) This module has submodules

- Valuations [Valuation](#)
- Proofs [Proofs](#)

Module	Contents
captureDcpo.lean	Engeler reflexive dcpo used to witness capture.
capturePair.lean	Injective pairing $P_{\text{fin}}(\mathbb{N}) \times \mathbb{N} \rightarrow \mathbb{N}$.
captureListCode.lean	List encoding used to inject $\text{List } \mathbb{N}$ into $\mathbb{N}$.

Module	Contents
capturePairLaws.lean	Proves ‘captureListCode_injective’, ‘capturePair_injective’.
captureVal.lean	Defines ‘captureVal’: Valuation (Fin 2) (Set ℕ).
chiBundle.lean	Pointwise union of Boolean values of χ_S along a set of numerals.
engelerWithNumerals.lean	Church Booleans and numerals on the Engeler model.
interp.lean	Defines ‘interp’.
interpClosed.lean	Closed-term interpretation $\llbracket M \rrbracket := \llbracket M \rrbracket_\emptyset$.
churchInterpLaws.lean	Proves ‘interp_closed_of_fv_empty’, ‘churchTrue_interp_eq’, ‘churchFalse_interp_eq’, ‘churchTrue_interp_ne_churchFalse’, and 5 further declarations.
numeralSuccGraph.lean	Successor graph used to separate Church numerals.
interpAgree.lean	Interpretations agree when valuations agree on $\text{fv}(M)$.
interpAbsApp.lean	Proves ‘interp_update_scott’, ‘interp_abs_app’.
scottContinuousBasics.lean	Proves ‘scottContinuous_eval’, ‘directedOn_image_monotone’, ‘scottContinuous_apply’, ‘scottContinuous_of_pi’, and 1 further declarations.
churchBoolSepWitness.lean	Witness pair $(\emptyset, \text{pair}(\{0\}, 0))$ used to separate Church Booleans.
churchNot.lean	Church negation $\lambda b. b \perp \top$ on Fin 2.
churchNotGraph.lean	Defines ‘churchNotGraph’: Set ℕ.
gbar.lean	Defines ‘gbar’: Set ℕ.
numeralFingerprint.lean	Defines ‘numeralFingerprint’: Set ℕ.
lemma_35_boolOpenBot.lean	Scott-open containing $\llbracket \perp \rrbracket$ but not $\llbracket \top \rrbracket$.
lemma_35_boolOpenTop.lean	Defines ‘lemma_35_boolOpenTop’: Set (Set ℕ).
lemma_35_numeralOpen.lean	Scott-open $U_m = \{X \mid m \in \Phi(X)\}$ containing $\llbracket c_m \rrbracket$ and no other numeral.

Valuations Valuation

Module	Contents
default.lean	Defines ‘default’: Valuation Var D.
empty.lean	The empty valuation $\emptyset$. Dummy values are unused on closed terms.
lookup.lean	The paper’s $\rho(x)$, given $x \in \text{dom}(\rho)$.
update.lean	The paper’s update $\rho(x := d)$: domain $\text{dom}(\rho) \cup \{x\}$, overwrite at x.

Proofs Proofs

Module	Contents
Core.lean	Interpretation.

Module	Contents
CoreCont.lean	Proofs in this block, including ‘chiBundle_scott Continuous’, ‘chiBundle_singleton’, ‘gbar_numeral’ and 12 further declarations.

Internal domain [InternalDomain](#) This module has submodules

- Proofs [Proofs](#)

Module	Contents
directedDownB.lean	$\{e \in D \mid e \ll d\}$ is directed.
inWayBelowDownB.lean	$e \in D$ and $e \ll d$.
wayBelowSubsetB.lean	$d \ll e$ in the subset order (CSL §4.1).
subsetUnionB.lean	$e \subseteq \bigcup S$.
isDirectedSubsetB.lean	S is directed under $\subseteq$.
nonemptyB.lean	S is nonempty.
directedDownF.lean	$\{e \in D \mid e \ll d\}$ is directed.
subsetF.lean	$x \subseteq y$ as $\forall z (z \in x \rightarrow z \in y)$.
inWayBelowDownF.lean	$e \in D \wedge e \ll d$.
wayBelowSubsetF.lean	$d \ll e$ under $\subseteq$.
subsetUnionF.lean	$e \subseteq \bigcup S$ as $\forall x (x \in e \rightarrow \exists y (y \in S \wedge x \in y))$.
isDirectedSubsetF.lean	S is directed under $\subseteq$.
nonemptyF.lean	$\exists x. x \in S$.
existsMemB.lean	Semantic union membership: $\exists u (u \in X \wedge v \in u)$.
isBaseSubsetB.lean	B is a base for D under $\subseteq$.
isBaseSubsetF.lean	Defines ‘isBaseSubsetF’: SetFormula n.
isContinuousAtSubsetF.lean	Continuity of D at d .
joinsDownF.lean	$d = \bigcup \{e \in D \mid e \ll d\}$.
isCompleteLatticeSubsetB.lean	D is a complete lattice under $\subseteq$.
isSupSubsetB.lean	x is the $\subseteq$ -supremum of S .
isUpperBoundSubsetB.lean	x is a $\subseteq$ -upper bound of S .
isCompleteLatticeSubsetF.lean	D is a complete lattice under $\subseteq$.
isSupSubsetF.lean	x is the $\subseteq$ -supremum of S .
isUpperBoundSubsetF.lean	x is a $\subseteq$ -upper bound of S .
isContinuousAtSubsetB.lean	Continuity of D at d (subset order).
joinsDownB.lean	$d = \bigcup \{e \in D \mid e \ll d\}$.
isContinuousLatticeSubsetB.lean	D is a continuous lattice under $\subseteq$.
isContinuousLatticeSubsetF.lean	D is a continuous lattice under $\subseteq$.
sUnionB.lean	Defines ‘sUnionB’: AName.{u} A.
union2B.lean	Binary union name, as the disjoint union of domains.

Module	Contents
wayBelow_le_formula.lean	$\ll$ implies $\subseteq$, as a closed $\mathfrak{L}$ _Set sentence.

Proofs [Proofs](#)

Module	Contents
Core.lean	Semantic Boolean values (subset order).
CoreCont.lean	Semantic Boolean values (subset order).

Internal evaluation [InternalEval](#) This module has submodules

- Internal reflexive model [InternalReflexiveModel](#)
- Proofs [Proofs](#)

Module	Contents
infMemB0pairBLaws.lean	Proves ‘inf_memB_opairB_le_iSup_child’: $Y.idx, \text{mem } B \text{ (opairB } x \text{ (Y.child } j)) \text{ F}$.
relFunOnProdB.lean	Declares ‘relFunOnProdB’, ‘relFunOnProdB_val’.
prodBIdxEquiv.lean	The displayed index type of an internal Cartesian product.
oidProdBLaws.lean	Proves ‘oid_prodB_eq’, ‘oid_prodB_eps’.
prodBIdxOf.lean	Declares ‘prodBIdxOf’, ‘prodBIdxEquiv_idxOf’, ‘prodB_child_eq’, ‘prodB_child_idxOf’, and 2 further declarations.
IsRelElementAt.lean	A Boolean row representing one element of D at extent a.
applyRelOfElements.lean	Declares ‘applyRelOfElements’, ‘applyRelOfElements_val’.
relFunOfIsRelElementAt.lean	Declares ‘relFunOfIsRelElementAt’, ‘relFunOfIsRelElementAt_val’.
constantRowGraph.lean	Raw graph of a constant generalized element.
interpDKBodyGraph.lean	The internal graph of the meta-function in the abstraction clause.
interpDKRelVal.lean	Declares ‘interpDKRelVal’, ‘interpDKRelVal_var’, ‘interpDKRelVal_const’, ‘interpDKRelVal_app’.
prodOrderRelB.lean	Defines ‘prodOrderRelB’: $AName.\{u\} \text{ A}$.

Internal reflexive model [InternalReflexiveModel](#)

Module	Contents
evalGraph.lean	Defines ‘evalGraph’: $\text{InternalReflexiveModel } (A)$.
evalRel.lean	Defines ‘evalRel’: $\text{InternalReflexiveModel } (A)$.
memMapSpaceLaws.lean	Proves ‘mem_mapSpace_le_scottContinuous’, ‘mem_mapSpace_le_function’.

Module	Contents
validComponents.lean	Proves ‘valid_components’, ‘mapSpace_valid’, ‘fun_continuous’, ‘function_of_scottContinuous’, and 1 further declarations.

Proofs [Proofs](#)

Module	Contents
Core.lean	The recursive Boolean matrix.
CoreCont.lean	The six clauses packed by internal Definition 19.

Completed internal evaluation [InternalEvalComplete](#) This module has submodules

- Internal reflexive model [InternalReflexiveModel](#)
- Proofs [Proofs](#)

Module	Contents
graphImageB.lean	Defines ‘graphImageB’: $\text{AName}.\{u\} \text{ A}$.
degreePairSetoid.lean	Two-point setoid whose off-diagonal equality is the degree a .
interpDKAbsFamilyGraph.lean	The graph of the inner family $d \mapsto \llbracket P \rrbracket \rho[y:=d]$ as a map $D \rightarrow C$.
zeroPairGraph.lean	The empty $D \times D$ graph.
interpDKAbsFamilyImageLawsC.lean	Proves ‘interpDKAbsFamily_image_least’: Internal ReflexiveModel (A.
interpDKAbsFamilyImageLawsB.lean	A displayed inner graph is a Q-upper bound of the family image.
interpDKAbsFamilyImageLawsA.lean	Inner body graphs respect equality of the outer argument.

Internal reflexive model [InternalReflexiveModel](#)

Module	Contents
evalAtGraph.lean	Defines ‘evalAtGraph’: InternalReflexiveModel (A.
pointwiseSupGraph.lean	Defines ‘pointwiseSupGraph’: InternalReflexiveModel (A.

Proofs [Proofs](#)

Module	Contents
Core.lean	A generalized element transports its coefficients along target equality.
CoreCont.lean	A generalized element transports its coefficients along target equality.
CoreContCont.lean	A generalized element transports its coefficients along target equality.

Module	Contents
CoreContContCont.lean	Abstraction constructor.
CoreContContContCont.lean	A generalized element transports its coefficients along target equality.
CoreContContContContContCont.lean	A generalized element transports its coefficients along target equality.

Packed internal evaluation [InternalEvalPack](#) This module has submodules

- Internal reflexive model [InternalReflexiveModel](#)
- Proofs [Proofs](#)
- de Bruijn terms [LamDK](#)

Module	Contents
asLamDK.lean	AName.idx is not unfolded by rewrite, so we reify the lamDKB carrier.
LamDK.lean	Module ‘LamDK’.
OidSeparated.lean	Defines ‘OidSeparated’: Prop.
asLam.lean	Declares ‘asLam’, ‘asLam_encode’.
LamDKLaws.lean	Proves ‘encodeLamDKB_ofLam’, ‘encodeLamDKB_ofLam_asLam’, ‘lamDKVal_ofLam’.

Internal reflexive model [InternalReflexiveModel](#)

Module	Contents
interpDKGraph.lean	Defines ‘interpDKGraph’: InternalReflexiveModel (A.
interpDKRel.lean	Relational function assembled from the term-indexed evaluation rows.
interpDKName.lean	Mix a Definition 25 row to a D-index via completeness.
interpDKPureGraph.lean	Defines ‘interpDKPureGraph’: InternalReflexiveModel (A.
interpDKPureRel.lean	Defines ‘interpDKPureRel’: InternalReflexiveModel (A.

Proofs [Proofs](#)

Module	Contents
Core.lean	Generalized-element restriction.
CorePost.lean	Evaluator name.
CoreCont.lean	Definition 25 graph clauses (continued).
CoreContCont.lean	Generalized-element restriction.

Module	Contents
weight.lean	Defines ‘weight’.

A.3.2 The Engeler model (section 4.2) [Engeler](#)

This module has submodules

- Internal Theorem 30 [Theorem30Internal](#)

Module	Contents
Engeler.lean	The Engeler model (Proposition 29) and numerals (Definition 32).
EngelerVA.lean	Theorem 30: the Engeler model in V^A . The paper (CSL 2026, before Proposition 29) takes a set E with an injective structure map $P_fin(E) \times E \rightarrow E$. The initial algebra of $P_fin \times Id$ is empty and is not used.
LambdaConstVA.lean	Constant-bearing lambda syntax in V^A . This is Example 21 and Definition 20 with the paper’s fourth tag (3,d). Unlike the pure checked-variable presentation, an arbitrary Boolean-valued constant set need not have extent $\top$; term extents are therefore propagated recursively through the inductive constructors.
LambdaVA.lean	Pure λ -syntax in V^A (Example 21 at V^A , Proposition 22, Example 24). Furber–Mardare–Panangaden–Scott, CSL 2026, Example 21, Proposition 22, and Example 24. Pure terms are encoded as pre-sets by the paper’s tags: (0,x) a variable, (1,(x,M)) an abstraction, (2,(M,N)) an application.
Theorem30Internal.lean	Internal Theorem 30: Definition 19 for the Engeler model in V^A . Paper Theorem 30: $(P^A(\text{check } E), \subseteq, \cdot, \text{lam})$ is a reflexive dcpo in V^A . The exact Lean form is <code>isReflexive DcpoB ... = $\top$</code> (Definition 19). The existing <code>theorem_30 / engelerVA</code> package remains the weaker external/canonical-carrier statement.
Lemma31.lean	Lemma 31: check commutation of the Engeler interpretation. Furber–Mardare–Panangaden–Scott, CSL 2026, Lemma 31. For a pure term $M \in \Lambda(\text{Var})$ and a valuation ρ with $\text{fv}(M) \subseteq \text{dom}(\rho)$.
Corollary34.lean	Corollary 34: reflexive continuous lattice with numerals in V^A . Furber–Mardare–Panangaden–Scott, CSL 2026, Corollary 34. The paper says the Engeler model in V^A (Theorem 30) is a reflexive continuous lattice with numerals, given check of the Church Booleans and numerals, with the whole statement interpreted in V^A .

Internal Theorem 30 [Theorem30Internal](#) This module has submodules

- Proofs [Proofs](#)

Module	Contents
engelerAppGraph.lean	Internal graph of $X \mapsto F \cdot X$.
engelerD.lean	Defines ‘engelerD’: $\text{AName}.\{u\} \text{ A}$.
engelerAppRel.lean	Relational application of a name F as a self-map of engelerD.
engelerAppIdx.lean	Index-level application of a name F on the Engeler carrier.
engelerC.lean	Scott-continuous self-maps of the Engeler carrier.
engelerR.lean	The subset order on the Engeler carrier.
subsetOrderRelB.lean	Defines ‘subsetOrderRelB’: $\text{AName}.\{u\} \text{ A}$.
subsetPairPred.lean	Characteristic predicate of a pair related by $\subseteq$.
engelerGroundFinPred.lean	Defines ‘engelerGroundFinPred’: A .
engelerGroundFins.lean	Defines ‘engelerGroundFins’: $\text{AName}.\{u\} \text{ A}$.
engelerQ.lean	Pointwise order on the Engeler continuous-map space.
pointwiseOrderPred.lean	Defines ‘pointwiseOrderPred’: A .
engelerFun.lean	Internal graph of $F \mapsto (X \mapsto F \cdot X)$.
engelerFunRel.lean	Defines ‘engelerFunRel’: $\text{RelFun} \text{ (oid (engelerD (A$
engelerFunIdx.lean	Index-level $F \mapsto (X \mapsto F \cdot X)$ as a map $D \rightarrow C$.
engelerLamGraphName.lean	Defines ‘engelerLamGraphName’: $\text{AName}.\{u\} \text{ A}$.
engelerLamGraphPred.lean	Defines ‘engelerLamGraphPred’: A .
engelerLamIdx.lean	Defines ‘engelerLamIdx’: $(\text{engelerC (A$
engelerLamB.lean	Defines ‘engelerLamB’: $\text{AName}.\{u\} \text{ A}$.
engelerLamRel.lean	Defines ‘engelerLamRel’: $\text{RelFun} \text{ (oid (engelerC (A$
theorem_30_internalModel.lean	Defines ‘theorem_30_internalModel’: $\text{InternalReflexive Model (A}$.

Proofs Proofs

Module	Contents
AppIdx.lean	Carrier D.
FunIdx.lean	Carrier D.
Core.lean	Carrier D.
LamIdx.lean	Abstraction graph $\text{Lam} : C \rightarrow D$.
CoreContContContCont.lean	Carrier D.
CoreCont.lean	Proofs in this block, including ‘isFunctionB_engelerLam B’, ‘subsetB_engelerAppName_mono_fun’, ‘engelerFun Rel_val’ and 5 further declarations.
CoreContCont.lean	Carrier D.
CoreContContCont.lean	Carrier D.

A.3.3 Injective spaces and oracles (section 4.3) Oracles

Module	Contents
ReflexiveDcpoWithNumerals.lean	Definition 32: a reflexive dcpo with numerals. We record the algebraic interface (distinct Booleans, if/succ/pred/0?) without a particular encoding of closed terms as domain elements.
NumeralSeparators.lean	Denotations of negation and the derived $m?$ tests used in the proof of Lemma 35(i).
MapsNumerals.lean	Closed M sending each Church numeral to a Church numeral.
Lemma35General.lean	Lemma 35 for arbitrary reflexive dcpos with numerals. The paper derives numeral-separating closed terms from Definition 32. NumeralSeparators records their denotations.
Prop36.lean	Proposition 36: many-one comparison via a numeral-to-numeral combinator. Furber–Mardare–Panangaden–Scott, CSL 2026, Proposition 36. <code>proposition_36_</code> is the paper type on an arbitrary reflexive continuous lattice with Church numerals (<code>churchWithNumerals</code>).

A.4 Random variables [RandomVariables](#)

Section 5 of the paper.

This module has submodules

- Domain-valued random variables [Random](#)
- Negligibility spaces [NegligibilitySpace](#)
- Coin space [Coin](#)

Module	Contents
NegligibilitySpace.lean	A negligibility space (paper §5): a measurable space with a σ -ideal N such that Σ / N is complete, not merely σ -complete. Completeness is packaged as existence of essential suprema.
Random.lean	Domain-valued random variables (Definitions 37–41, Propositions 39–40). $\mathcal{L}^0(X; \mathcal{P}(Y))$ is the set of measurable maps $X \rightarrow \text{Set } Y$. Quotienting by a σ -ideal of negligible sets yields an $A(X)$ -poset (Lemma 38). G_X is the strict $A(X)$ -poset isomorphism of Proposition 39; application is the homomorphism of Proposition 40.
Coin.lean	Coin space (Proposition 42, Theorem 43). Independent fair coins on $2^\omega \times 2^\omega$. <code>coinS1</code> / <code>coinS2</code> are equation (5): $S_i(\text{check } n) = [D_ \{i,n,1\}]$ in $\mathcal{P}^\wedge\{A(X)\}(\text{check } \omega)$ with $A(X) = \text{coinAlgebra}$. <code>proposition_42</code> is the raw measure-theoretic Boolean-value-0 core (<code>coinMeasure _ = 0</code>).

A.4.1 Domain-valued random variables [Random](#)

This module has submodules

- Associated Boolean algebra [AssociatedAlgebra](#)
- L^0 equivalence [L0](#)
- Negligibility ideal [NegligibilitySpace](#)
- Measure algebra [MeasureAlgebra](#)
- Proofs [Proofs](#)

Module	Contents
AssociatedAlgebra.lean	The complete Boolean algebra associated to a negligibility space.
aeSetoid.lean	Defines ‘aeSetoid’: Setoid (Set X).
aeEq.lean	Declares ‘aeEq’, ‘aeEq_refl’, ‘aeEq_symm’, ‘aeEq_trans’, and 5 further declarations.
symmDiff.lean	Proves ‘symmDiff_triangle_subset’, ‘union_symmDiff_subset’, ‘inter_symmDiff_subset’, ‘sdiff_symmDiff_subset’.
G_X.lean	Equation (4) on the a.e. quotient: $G_X([a])(y) = [a^{-1}(B_y)]$.
G_pre.lean	Lemma 41 (pre-quotient): G_{pre} of a constant random variable is the check-set.
posBasic.lean	Positive / Scott subbasic sets $B_y = \{S \mid y \in S\}$ (paper §5).
constRV.lean	Lemma 41 shape: the constant random variable K_S has check-image S .
IsL0.lean	Defines ‘IsL0’: Prop.
l0App.lean	Pointwise Engeler application on random variables.
l0AppLaws.lean	Proves ‘l0App_preimage’, ‘l0App_isL0’, ‘l0App_ae’.
l0Eq.lean	Definition 37: $\mathcal{L}^0$ -valued equality.
l0AE.lean	Declares ‘l0AE’, ‘l0AE_refl’, ‘l0AE_symm’, ‘l0AE_trans’.
L0Fun.lean	Measurable random variables, before a.e. quotient.
engelerAppA.lean	Defines ‘engelerAppA’: $\text{ASubset } A \text{ } E$.
G_preAeEq.lean	Proves ‘G_pre_aeEq’: $\text{aeEq } N (G_{\text{pre}} \text{ a.val } y) (G_{\text{pre}} \text{ b.val } y)$.
L0.lean	Paper $L^0(X; \mathcal{P}(Y))$: measurable maps modulo a.e. equality.
l0Setoid.lean	Defines ‘l0Setoid’: Setoid ($L0\text{Fun } X \text{ } Y$).
G_X_inv.lean	Inverse of G_X : $a(x) = \{y \mid x \in \text{measurable representative of } b(y)\}$.
G_X_measure.lean	Equation (4) on MeasureAlgebra: $G_X([a])(y) = [a^{-1}(B_y)]$.
MeasureAlgebra.lean	Paper $A(X) = \Sigma / N(\mu)$.
MeasurableSetoid.lean	Defines ‘MeasurableSetoid’: $\text{Set } X // \text{MeasurableSet } s$.

Module	Contents
G_preAeEqMeasure.lean	Proves ‘G_pre_aeEq_measure’: μ (symmDiff (G_pre a.val y) (G_pre b.val y)) = 0.
l0AE_measure.lean	Declares ‘l0AE_measure’, ‘l0AE_measure_refl’, ‘l0AE_measure_symm’, ‘l0AE_measure_trans’.
L0Measure.lean	Paper $L^0(X; \mathcal{P}(Y))$ on $A(X) = \Sigma / N(\mu)$.
l0Setoid_measure.lean	Defines ‘l0Setoid_measure’: Setoid (L0Fun X Y).
G_X_measure_inv.lean	Defines ‘G_X_measure_inv’: L0Fun X Y.
IsAtomic.lean	Defines ‘IsAtomic’: Prop.
l0LeAeEq.lean	Proves ‘l0Le_subset_of_ae’, ‘l0Le_aeEq’.
l0Le.lean	Definition 37: $\mathcal{L}^0$ -valued order.
l0AppAeMeasure.lean	Proves ‘l0App_ae_measure’: l0AE_measure μ (l0App pair a.val b.val, l0App_isL0 pair a.property b.property) (l0App pair a'.val b'.val, l0App_isL0 pair a'.property b'.property).
proposition_39.lean	Proposition 39: G_X is a strict isomorphism of A(X)-posets.
l0Poset.lean	Lemma 38: L^0 is an A(X)-poset.
G_XInvLaws.lean	Proves ‘G_X_le’, ‘G_X_inv_right’, ‘G_X_inv_left’.
G_X_mk.lean	Proves ‘G_X_mk’: $G_X N (L0.mk N a) y = \text{AssociatedAlgebra.mk } N (G_pre a.val y)$.
G_X_measure_invLaws.lean	Inverse of G_X_measure: $a(x) = \{y \mid x \in \text{out}(b y)\}$.
l0LeAeEqMeasure.lean	Proves ‘l0Le_aeEq_measure’: μ (symmDiff (l0Le a.val b.val) (l0Le a'.val b'.val)) = 0.
atomlessSeq.lean	Defines ‘atomlessSeq’: $\mathbb{N} \rightarrow A$.
atomlessSeqAux.lean	Defines ‘atomlessSeqAux’: $A, x \neq \perp \wedge (\forall b, \text{IsAtom } b \rightarrow \neg b \leq x) \mid 0 \Rightarrow \text{let } h$.
existsLtAtomless.lean	Proves ‘exists_pos_atomless_of_not_isAtomic’, ‘exists_lt_atomless’.
l0APoset.lean	Lemma 38, unquotiented: $\mathcal{L}^0$ is an A-poset for $A = \text{Set } X$.
l0Poset_measure.lean	Lemma 38 on MeasureAlgebra: L^0 is an A(X)-poset.
meetlessSeq.lean	Defines ‘meetlessSeq’: $\mathbb{N} \rightarrow A$.
proposition_39_measure.lean	Proposition 39 on MeasureAlgebra: G_X is a strict A(X)-poset iso.
G_X_measureLe.lean	Proves ‘G_X_measure_le’: subsetB (G_X_measure μ a) (G_X_measure μ b) = L0.le_measure μ a b.
paperRV.lean	Paper L^0 / G_X on MeasureAlgebra ($A(X) = \Sigma / N$). The un-suffixed G_X / lemma_38 / proposition_39 / proposition_40 / lemma_41_algebra remain the all-sets AssociatedAlgebra forms. These _measure names are the paper maps on MeasureAlgebra μ .

Associated Boolean algebra [AssociatedAlgebra](#)

Module	Contents
Le.lean	Order: $[s] \leq [t]$ iff $s \setminus \{t\}$ is negligible.
mk.lean	Declares ‘mk’, ‘mk_eq_iff’, ‘mk_out’, ‘aeEq_out’.
bot.lean	Defines ‘bot’: AssociatedAlgebra N.
compl.lean	Defines ‘compl’: AssociatedAlgebra N.
inf.lean	Declares ‘inf’, ‘inf_mk’.
sInf.lean	Defines ‘sInf’: AssociatedAlgebra N.
sSup.lean	Defines ‘sSup’: AssociatedAlgebra N.
sup.lean	Defines ‘sup’: AssociatedAlgebra N.
top.lean	Defines ‘top’: AssociatedAlgebra N.
instances.lean	Proves ‘le_refl’, ‘le_trans’, ‘le_antisymm’, ‘inf_le_left’, and 21 further declarations.
himpMk.lean	Proves ‘himp_mk’: $\text{mk } N \ s \Rightarrow \text{mk } N \ t = \text{mk } N \ (s^c \cup t)$.
supMk.lean	Proves ‘sup_mk’, ‘iSup_mk’.
complMk.lean	Proves ‘mk_compl’, ‘compl_mk’.
iInfMk.lean	Proves ‘iInf_mk’, ‘iInf_mk_finset’.
measRepMk.lean	Proves ‘measRep_aeEq’, ‘mk_measRep’.

L^0 equivalence [L0](#)

Module	Contents
mk.lean	Defines ‘mk’: $L^0 \ N \ Y$.
mk_measure.lean	Defines ‘mk_measure’: $\text{_root_}.\text{Scott2026.L0Measure } \mu \ Y$.
app.lean	Quotiented pointwise application $[a] \cdot [b] = [a \cdot b]$.
le.lean	Defines ‘le’: AssociatedAlgebra N.
app_measure.lean	Declares ‘app_measure’, ‘app_measure_mk’.
equivPower.lean	G_X as a type equivalence (Proposition 39).
leTrans.lean	Proves ‘le_trans’, ‘le_le_refl’.
equivPower_measure.lean	G_X as a type equivalence on the paper algebra $A(X) = \Sigma/N(\mu)$.
le_measure.lean	Declares ‘le_measure’, ‘le_measure_mk’.
leTransMeasure.lean	Proves ‘le_trans_measure’, ‘le_le_refl_measure’.

Negligibility ideal [NegligibilitySpace](#)

Module	Contents
measRep.lean	Declares ‘measRep’, ‘measRep_measurable’, ‘measRep_symmDiff’.
unionCountable.lean	Proves ‘union_countable’: $N.\text{negligible } (\bigcup i, s \ i)$.
ofMeasure.lean	Defines ‘ofMeasure’: NegligibilitySpace X.
ofMeasureLaws.lean	Flatten a maximizing sequence of countable subfamilies.

Module	Contents
--------	----------

Measure algebra [MeasureAlgebra](#)

Module	Contents
mk.lean	Declares ‘mk’, ‘mk_eq_iff’, ‘mk_congr’, ‘mk_out’.
outAe.lean	Proves ‘mk_out_ae’, ‘mk_out_ae_of’, ‘measAe_symm’, ‘measAe_union’, and 4 further declarations.
Le.lean	Order: $[s] \leq [t]$ iff $\mu (s \setminus \{t\}) = 0$.
bot.lean	Defines ‘bot’: $\text{_root_}.\text{Scott2026.MeasureAlgebra } \mu$.
compl.lean	Declares ‘compl’, ‘compl_mk’.
inf.lean	Defines ‘inf’: $\text{_root_}.\text{Scott2026.MeasureAlgebra } \mu$.
sInf.lean	Defines ‘sInf’: $\text{_root_}.\text{Scott2026.MeasureAlgebra } \mu$.
sSup.lean	Defines ‘sSup’: $\text{MeasureAlgebra } \mu$.
sSupWitness.lean	Proves ‘sSupWitness’: $\text{_root_}.\text{Scott2026.MeasureAlgebra } \mu).val \setminus \{v\} = 0 \rightarrow \mu (u \setminus \{v\}) = 0$.
sup.lean	Declares ‘sup’, ‘sup_mk’.
toAssociated.lean	Defines ‘toAssociated’: $\text{_root_}.\text{Scott2026.MeasureAlgebra } \mu \rightarrow \text{AssociatedAlgebra } (\text{NegligibilitySpace.of Measure } \mu \text{ hN})$.
top.lean	Defines ‘top’: $\text{_root_}.\text{Scott2026.MeasureAlgebra } \mu$.
instances.lean	Proves ‘le_refl’, ‘le_trans’, ‘le_antisymm’, ‘inf_le_left’, and 23 further declarations.
himpMk.lean	Proves ‘himp_mk’: $\text{mk } \mu \text{ s hs} \Rightarrow \text{mk } \mu \text{ t ht} = \text{mk } \mu (s^c \cup t) (\text{hs.compl.union ht})$.
supMk.lean	Proves ‘iSup_mk’: $(\bigcup i, \text{mk } \mu (s i) (\text{hs } i)) = \text{mk } \mu (\bigcup i, s i) (\text{MeasurableSet.iUnion hs})$.
iInfMkFinset.lean	Proves ‘iInf_mk_finset’: $(\bigcap k \in K, \text{mk } \mu (s k) (\text{hs } k)) = \text{mk } \mu (\bigcap k \in K, s k) (\text{Finset.measurableSet_biInter } K \text{ fun } k _ \Rightarrow \text{hs } k)$.
infMk.lean	Proves ‘inf_mk’: $\text{inf } \mu (\text{mk } \mu \text{ s hs}) (\text{mk } \mu \text{ t ht}) = \text{mk } \mu (s \cap t) (\text{hs.inter ht})$.
iffMk.lean	Proves ‘iff_mk’, ‘iInf_mk’.

Proofs [Proofs](#)

Module	Contents
Core.lean	Module ‘Core’.
CoreCont.lean	Module ‘CoreCont’.
CoreContCont.lean	Definition 37: measurable maps into Set Y (Borel of the positive topology). We take measurability of all preimages of B_y, which generate that σ -algebra when Y is countable.

A.4.2 Negligibility spaces `NegligibilitySpace`

Module	Contents
<code>union2.lean</code>	Proves ‘union’.

A.4.3 Coin space `Coin`

This module has submodules

- Proofs `Proofs`

Module	Contents
<code>Cantor.lean</code>	Defines ‘Cantor’.
<code>CoinSpace.lean</code>	Defines ‘CoinSpace’.
<code>D1.lean</code>	Defines ‘D1’: Set CoinSpace .
<code>D2.lean</code>	Defines ‘D2’: Set CoinSpace .
<code>G_X_coin.lean</code>	Defines ‘G_X_coin’: $\text{Type}^* \{ \text{Countable } Y \}$.
<code>coinMeasure.lean</code>	Defines ‘coinMeasure’: Measure CoinSpace .
<code>cantorMeasure.lean</code>	Defines ‘cantorMeasure’: Measure Cantor .
<code>fairBit.lean</code>	Defines ‘fairBit’: Measure Bool .
<code>halfI.lean</code>	Defines ‘halfI’: I .
<code>agreeBit.lean</code>	Defines ‘agreeBit’: Set CoinSpace .
<code>agreeSet.lean</code>	Defines ‘agreeSet’: Set CoinSpace .
<code>agreeSet_swap.lean</code>	Defines ‘agreeSet_swap’: Set CoinSpace .
<code>agreeSlice.lean</code>	Defines ‘agreeSlice’: Set CoinSpace .
<code>appChi1L0.lean</code>	Defines ‘appChi1L0’: $\text{L0Fun CoinSpace } \mathbb{N}$.
<code>coinChi1_isL0.lean</code>	Proves ‘coinChi1_isL0’: IsL0 coinChi1 .
<code>coinChi1.lean</code>	Defines ‘coinChi1’: $\text{CoinSpace} \rightarrow \text{Set } \mathbb{N}$.
<code>bits1.lean</code>	Defines ‘bits1’: $\text{Set } \mathbb{N}$.
<code>measurableSet_chiOracle.lean</code>	Proves ‘mem_gbar_iff’, ‘mem_chiOracle_iff’, ‘mem_chiNum_bits1’, ‘mem_chiNum_bits2’, and 6 further declarations.
<code>bits2.lean</code>	Defines ‘bits2’: $\text{Set } \mathbb{N}$.
<code>measurableSet_D1.lean</code>	Proves ‘measurableSet_D1’: $\text{MeasurableSet } (D1 \text{ n b})$.
<code>measurableSet_D2.lean</code>	Proves ‘measurableSet_D2’: $\text{MeasurableSet } (D2 \text{ n b})$.
<code>appChi2L0.lean</code>	Defines ‘appChi2L0’: $\text{L0Fun CoinSpace } \mathbb{N}$.
<code>coinChi2_isL0.lean</code>	Proves ‘coinChi2_isL0’: IsL0 coinChi2 .
<code>coinChi2.lean</code>	Defines ‘coinChi2’: $\text{CoinSpace} \rightarrow \text{Set } \mathbb{N}$.
<code>appCoin.lean</code>	Defines ‘appCoin’: $\text{ASubset coinAlgebra } \mathbb{N}$.
<code>coinAlgebra.lean</code>	Paper A(X) for fair-coin space: measurable sets modulo null.
<code>boolValSet1.lean</code>	Defines ‘boolValSet1’: Set CoinSpace .

Module	Contents
coinA1Fun.lean	Paper a_1 with $[a_1] = G_X^{\{-1\}}(d_1)$.
coinD1.lean	Paper $d_1 \in P^A$ (check E): G_X of the fiberwise Lemma 35(ii) oracle.
coinChi1Fun.lean	Defines ‘coinChi1Fun’: $L0Fun$ $CoinSpace$ $\mathbb{N}$.
boolValSet2.lean	Defines ‘boolValSet2’: Set $CoinSpace$.
coinA2Fun.lean	Paper a_2 with $[a_2] = G_X^{\{-1\}}(d_2)$.
coinD2.lean	Paper $d_2 \in P^A$ (check E).
coinChi2Fun.lean	Defines ‘coinChi2Fun’: $L0Fun$ $CoinSpace$ $\mathbb{N}$.
coinL0.lean	Defines ‘coinL0’.
coinS1.lean	Paper equation (5): $S_1(\text{check } n) = [D_ \{1,n,1\}]$ in $P^{\{A(X)\}}(\text{check } \omega)$.
coinS2.lean	Paper equation (5): $S_2(\text{check } n) = [D_ \{2,n,1\}]$ in $P^{\{A(X)\}}(\text{check } \omega)$.
constL0.lean	Defines ‘constL0’: $L0Fun$ $CoinSpace$ $\mathbb{N}$.
finitePreimageSet.lean	Defines ‘finitePreimageSet’: Set $CoinSpace$.
finitePreimageSet_swap.lean	Defines ‘finitePreimageSet_swap’: Set $CoinSpace$.
imageBit.lean	Defines ‘imageBit’: $Bool$.
invAppFun.lean	Defines ‘invAppFun’: $L0Fun$ $CoinSpace$ $\mathbb{N}$.
mapsAgreeSet.lean	Defines ‘mapsAgreeSet’: Set $CoinSpace$.
mapsAgreeSet_swap.lean	Defines ‘mapsAgreeSet_swap’: Set $CoinSpace$.
mixedCyl.lean	Defines ‘mixedCyl’: Set $CoinSpace$.
mixedCylSub.lean	Defines ‘mixedCylSub’: Set $CoinSpace$.
paperGoodSet.lean	Defines ‘paperGoodSet’: Set $CoinSpace$.
paperReductionNull.lean	Defines ‘paperReductionNull’: Set $CoinSpace$.
reductionNullSet.lean	Defines ‘reductionNullSet’: Set $CoinSpace$.
vaSubset.lean	Membership of an Engeler carrier name as an A-subset of $\mathbb{N}$.

Proofs Proofs

Module	Contents
Core.lean	Coin space (Proposition 42, Theorem 43). Independent fair coins on $2^{\omega} \times 2^{\omega}$. $\text{coinS1} / \text{coinS2}$ are equation (5): $S_i(\text{check } n) = [D_ \{i,n,1\}]$ in $\mathcal{P}^{\{A(X)\}}(\text{check } \omega)$ with $A(X) = \text{coinAlgebra. proposition_42}$ is the raw measure-theoretic Boolean-value-0 core ($\text{coinMeasure } _ = 0$).
CoreCont.lean	Coin space (Proposition 42, Theorem 43). Independent fair coins on $2^{\omega} \times 2^{\omega}$. $\text{coinS1} / \text{coinS2}$ are equation (5): $S_i(\text{check } n) = [D_ \{i,n,1\}]$ in $\mathcal{P}^{\{A(X)\}}(\text{check } \omega)$ with $A(X) = \text{coinAlgebra. proposition_42}$ is the raw measure-theoretic Boolean-value-0 core ($\text{coinMeasure } _ = 0$).

Module	Contents
CoreContCont.lean	Coin space (Proposition 42, Theorem 43). Independent fair coins on $2^\omega \times 2^\omega$. <code>coinS1 / coinS2</code> are equation (5): $S_i(\text{check } n) = [D_{\{i,n,1\}}]$ in $\mathcal{P}^{\{A(X)\}}(\text{check } \omega)$ with $A(X) = \text{coinAlgebra}$. <code>proposition_42</code> is the raw measure-theoretic Boolean-value-0 core (<code>coinMeasure _ = 0</code>).

A.4.4 Whole-paper capstone [Paper.lean](#)

Module	Contents
Paper.lean	Whole-paper capstone (csl2026). <code>csl2026</code> is the Palomar-compared theorem: Theorem 43 as incomparability under <code>proposition_36_i</code> on <code>engelerWith</code> Numerals. Challenge states that same oracle relation with <code>engelerApp</code> and <code>engelerLam</code> .

Total: 462 modules, 51133 lines of Lean in `Scott2026/`.

Primary source (PDF): [sources/Scott2026.pdf](#) — Furber, Mardare, Panangaden, and Scott, *Interpreting Lambda Calculus in Domain-Valued Random Variables* (LIPIcs, CSL 2026).